第三节：Jinja2模板

一、模板的基本使用

二、渲染变量

三、控制语句

1.if语句

2.for语句

四、模板继承

五、加载静态文件

六、感悟

1.对jinja2继承、css隔离小理解：

在开发过程中，用户访问某一个URL，服务器需要返回一个美观的界面，但写界面的HTML代码是及其庞大的，不能大量内嵌在python代码中，所以使用jinja2就可以达到将HTML和python分开的目的。

个人理解：jinja2可以在HTML中预留位置，将后端处理好的数据直接注入HTML，达到代码简洁的目的。

一、模板的基本使用

在Flask项目中，会有一个 `templates` 文件夹，只要没修改默认查找路径，默认寻找该文件夹下的模板文件。模板文件可以是任意纯文本文件，例如：.txt，.html等，为了规范，统一使用HTML文件。

代码如下：

app.py Python

```python
from flask import Flask,render_template

app = Flask(__name__)

@app.route("/")
def show():
    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True)
```

index.html代码为：

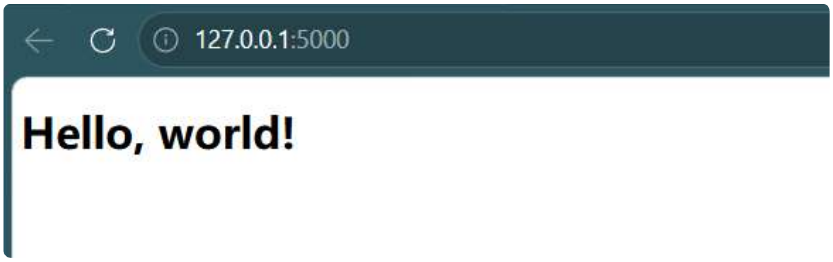
index.html HTML

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
      content="width=device-width, user-scalable=no, initial-scale=1.0, maximum-scale=1.0, minimum-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
  </head>
  <body>

    <p>Hello, world!</p>

  </body>
</html>
```

启动 `app.py` 后，在浏览器可以看到：



二、渲染变量

在HTML文件中，有些数据需要从数据库中动态读取，所以在渲染变量的过程中，应该如何使用变量？

代码如下：

app.py Python

```python
# 使用jinja2读取URL中的name注入HTML文件中
@app.route("/blog")
def blog():
    name = request.args.get('name') # 获取姓名
    return render_template("blog.html", name = name)# 把从URL获取的姓名插入HTML中
```

HTML代码：

HTML

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
      content="width=device-width, user-scalable=no, initial-scale=1.0, maximum-scale=1.0, minimum-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
  </head>
  <body>
    <h1>Hello {{name}} ! This is my blog</h1> /* 将上面在URL中获取的name注入到这里 */
  </body>
</html>
```

当然我们可以使用字典形式来注入数据，这样更加优雅：

app.py Python

```python
@app.route("/blog")
def blog():
    name = request.args.get('name')
    xinxi = {"sex":"男","age":18} # 直接写入字典
    return render_template("blog.html",name = name,xinxi = xinxi)
```

blog.html HTML

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
      content="width=device-width, user-scalable=no, initial-scale=1.0, maximum-scale=1.0, minimum-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
  </head>
  <body>
    <h1>Hello {{name}} ! This is my blog</h1>
    <p>age: {{xinxi.age}}</p> /*这里注意，由于后端传的是字典，所以这里想要注入的话需要使用 “字典名.键名” 形式*/
    <p>sex: {{xinxi["sex"]}}</p> /*当然，经典的取值也是可以的*/
  </body>
</html>
```

当然，如果创建一个对象的话，也是可以的，这样更加符合实际需要：

app.py Python

```python
class User:
    def __init__(self,name, sex, age):
        self.name = name
        self.sex = sex
        self.age = age


@app.route("/blog")
def blog():
    user = User("wedeout","男","18")
    return render_template("blog.html",user=user)
```

blog.html HTML

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
      content="width=device-width, user-scalable=no, initial-scale=1.0, maximum-scale=1.0, minimum-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
  </head>
  <body>
    <h1>Hello {{user.name}} ! This is my blog</h1> /* 这里注意，由于传入类，所以使用这种写法*/
    <p>age: {{user.age}}</p>
    <p>sex: {{user.sex}}
  </body>
</html>
```

但是，以上写法都太过冗余，频繁的写键值对太麻烦，不够优雅，所以我们引入以下写法：

app.py Python

```python
class User:
    def __init__(self,name, sex, age):
        self.name = name
        self.sex = sex
        self.age = age

@app.route("/blog")
def blog():
    user = User("wedeout","男","18")
    city = "上海"
    data = {
        "user":user,
        "city":city
    }
    return render_template("blog.html", **data) # **data 字典解包
```

在最后用一个字典将所有需要注入的变量封装起来并统一命名，使用时可以直接通过键来访问值的对象，字典真方便吧，我认为这个和json非常像。

三、控制语句

jinja2和python很像，但jinja2的控制语句都放在 `{% %}` 中，在控制语句结束后，需要加入对应的结束语句。

1.if语句

和python很像，可以使用 `>、<、>=、<=、==、!=` 来判断，也可以通过 `and、or、not` 进行逻辑操作。代码如下：

app.py Python

```python
# if控制语句练习
@app.route("/if")
def if_test():
    '''
    如果直接设定age=18，不会报错，但如果想要获取URL中的age，就需要将其转化为int类型，
    在后面添加type=int
    '''
    age = request.args.get("age",type=int) # 需要转化为int类型，才能在jinja2中比较
    return render_template("if_test.html",age=age)
```

根据age的大小判断是否成年：

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Title</title>
  </head>
  <body>

    {% if age > 18 %}
    <div> 你年龄为{{age}}，已经成年！ </div>
    {% elif age == 18 %}
    <div> 你年龄为{{age}}，刚刚成年！ </div>
    {% else %}
    <div> 你年龄为{{age}}，没有成年！ </div>
    {% endif %} /* 结束语句 */

  </body>
</html>
```

2.for语句

jinja2的for循环也和python很像。代码如下：

```python
@app.route("/for")
def for_test():
    books = [{
        "name": "三国演义",
        "author": "罗贯中",
        "price": 100
    }, {
        "name": "水浒传",
        "author": "施耐庵",
        "price": 99
    }, {
        "name": "红楼梦",
        "author": "曹雪芹",
        "price": 101
    }, {
        "name": "西游记",
        "author": "吴承恩",
        "price": 102
    }]
    return render_template("book.html",books=books) # 传入列表
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    <table>
        <thead>
            <tr>
                <th>书名</th>
                <th>作者</th>>
                <th>价格</th>>
            </tr>>
        </thead>
        <tbody>
            {% for book in books %} # 循环写入表格
                <tr>
                    <td>{{book.name}}</td>
                    <td>{{book.author}}</td>
                    <td>{{book.price}}</td>
                </tr>>
            {% endfor %}
        </tbody>>
        </table>
</body>
</html>
```

但是，jinja2中没有 `continue` 和 `break` 语句。

四、模板继承

在实际生活中，会发现网页的某些东西是重合的（导航栏等），所以可以通过模板继承来实现。

代码如下：

app.py Python

```python
@app.route("/list")
def list():
    return render_template("base/list.html")

@app.route("/detail")
def detail():
    return render_template("base/detail.html")
```

父模板 HTML

```html
<!DOCTYPE html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8">
    <!-- 独立标题块：每个页面单独改标题 -->
    <title>{% block title %}默认首页标题{% endblock %}</title>
    <!-- 公共样式区 -->
    {% block style %}
    <style>
      * {margin: 0; padding: 0; box-sizing: border-box;}
      .header {background:#222; color:#fff; padding:15px;}
      .container {padding:20px;}
      .footer {background:#eee; text-align:center; padding:10px; margin-top:30px;}
    </style>
    {% endblock %}
  </head>
  <body>
    <!-- 1. 公共头部导航：所有页面共用，子模板默认不修改 -->
    <div class="header">
      <h3>网站公共导航栏 | 全站统一头部</h3>
    </div>

    <!-- 2. 核心内容块：子页面必须覆写 -->
    <div class="container">
      {% block content %}
      <!-- 父模板默认内容：子模板不重写就显示这段 -->
      <p>这是父模板默认主体内容</p>
      {% endblock %}
    </div>

    <!-- 3. 额外扩展块：可选覆写 -->
    {% block extra %}{% endblock %}

    <!-- 4. 公共底部：全站共用 -->
    <div class="footer">
      {% block footer %}
      © 2026 Flask Jinja2 模板继承 | 公共底部
      {% endblock %}
    </div>
  </body>
</html>
```

子模块-列表页 HTML

```html
<!-- 1. 固定语法：继承父模板，必须写在第一行 -->
{% extends 'base/base.html' %} <!-- 注意，这里路径要加引号，并且写 -->

<!-- 2. 覆写标题区块 -->
{% block title %}
列表页面 - 专属标题
{% endblock %}

<!-- 3. 覆写主体内容，完全替换父模板 content -->
{% block content %}
<h2>商品列表页面</h2>
<p>这是子页面独有的内容，完全覆盖父模板默认内容</p>
<ul>
  <li>手机</li>
  <li>电脑</li>
  <li>平板</li>
</ul>
{% endblock %}

<!-- 4. 新增额外区块内容（父模板为空，子模板拓展） -->
{% block extra %}
<div style="color:red; margin:10px 0;">
  这是额外拓展区域，只有列表页显示
</div>
{% endblock %}

<!-- 5. 局部修改底部：保留父模板内容 + 追加子页面内容 -->
{% block footer %}
{{ super() }}  <!-- super() 关键：保留父模板原本的底部内容 -->
<br>
<span>列表页专属底部说明</span>
{% endblock %}
```

子模块-详情页 HTML

```html
{% extends 'base/base.html' %}

{% block title %}
详情页面 - 产品详情
{% endblock %}

{% block content %}
<h2>产品详情页</h2>
<div style="border:1px solid #ccc; padding:15px; margin-top:10px;">
  <h4>产品名称：Flask 实战教程</h4>
  <p>价格：99 元</p>
  <p>介绍：学习 Flask 开发、Jinja2 模板、路由、数据库等全套内容</p>
  <button>立即购买</button>
</div>
{% endblock %}

{% block extra %}
<div style="color:blue; margin:10px 0;">
  🔥 详情页专属广告：限时折扣中！
</div>
{% endblock %}

{% block footer %}
{{ super() }} # 在父模板的基础上追加
<br>
<small>详情页技术支持：2026 Flask 开发团队</small>
{% endblock %}
```

五、加载静态文件

在一个网页中，除了HTML代码，还需要一些CSS、js文件才能更加的美观。可以使用以下代码导入这些文件：

HTML

```html
<link href="{{"static", filename="css/mytest.css"}}">
<!-- <link href="/static/css/mytest.css> 这个也一样->
```

在浏览器渲染出来后，查看源代码为下：

HTML

```html
<link href="/static/css/mytest.css>
```

六、感悟

1.对jinja2继承、css隔离小理解：

1. main标签是最外层容器：`<body><main> “ 这里可以放多个div” </mian></body>`

2. 在main中可以包含多个div标签，可以通过对每个div标签定义类名（不同html可以重名），就可以实现css样式隔离
3. 具体实现：可以在父模板中使用以下代码：

HTML

```html
<main class="container {% block page-name %}{% endblock %}">
  {% block content %}{% endblock %}
</main>
```

通过子模版重写main标签就可以实现css隔离。

例子：

假设父模板写为：

父模板 HTML

```html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <title>模板继承</title>
    <!-- 引入全局css -->
    <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
    <!--
        container：全局公共类
        {% block page_class %}：留给子页面覆写，作为「页面唯一隔离标识」
    -->
    <div class="container {% block page_class %}{% endblock %}">
        {% block content %}{% endblock %}
    </div>
</body>
</html>
```

子模版写为：

子模版1 HTML

```html
{% extends "base.html" %}

<!-- 给当前页面单独设置唯一标识 -->
{% block page_class %}page_list{% endblock %}

{% block content %}
  <h2 class="title">列表页标题</h2>
  <p class="text">我是列表页内容</p>
{% endblock %}
```

子模版2 HTML

```html
{% extends "base.html" %}

<!-- 不一样的唯一标识 -->
{% block page_class %}page_detail{% endblock %}

{% block content %}
  <h2 class="title">详情页标题</h2>
  <p class="text">我是详情页内容</p>
{% endblock %}
```

CSS为：

CSS

```css
/* 公共全局样式 */
.container {
  width: 800px;
  margin: 0 auto;
}

/* 只作用在 列表页 里面的 .title .text */
.page_list .title {
  color: red;
}
.page_list .text {
  color: #666;
}

/* 只作用在 详情页 里面的 .title .text */
.page_detail .title {
  color: blue;
}
.page_detail .text {
  color: green;
}
```

这里需要注意一点，CCS语法的特点：

- `.类a .类b`，这种写法的意思选取含有 `类a` 标签内部，所有带有 `类b` 的后代标签。
- `.类a.类b`，这种写法的意思是找同时拥有 `类a` 和 `类b` 两个的类名的标签
- `类a 类b`，在HTML中，这种写法代表使用空格分割两个类名，使这个标签同时带有 `a` 和 `b` 的类名，对应为CSS是 `.a.b`

所以可以通过 `<div class="container {% block page_class %}{% endblock %}">` 这种方式来隔离CSS的使用区间，更好的进行分隔。